

RouteMind AI - INDENG 164 Final Project

I Gy Chen

May 2026

1 Introduction

Company Context. RouteMind AI is an LLM model routing company serving customers in coding, tutoring, and knowledge domain. Our platform sends each incoming user prompt to the most suitable LLM from a curated pool, simultaneously minimizing API cost and maximizing response quality. Unlike Cursor’s manual model-selection or a naive policy, RouteMind AI dynamically routes each prompt.

Central Problem in LLM Routing. We operate over a pool of prompts $p \in \mathcal{P}$ and a candidate set of models $m \in \mathcal{M}$. The dataset contains 33 models evaluated on 240 prompts across four benchmarks (AIME, GPQA, LCB, MMLU-Pro), with 9 zero-cost open-source models.

Our goal is to find a small yet effective model pool and define a routing policy $f : \mathcal{P} \rightarrow \mathcal{M}$ that minimizes the expected cost while maximizing the expected correctness.

Why Two-Stage Stochastic Optimization? At deployment time, the model pool must be configured before any specific prompt is known, due to licensing agreements and API integration. Once a prompt arrives, we can make a the routing decision with the full prompt context. This is why a two-stage stochastic optimization modeling well suits our need:

- **Stage 1:** Select which models to include in the pool before observing any prompt.
- **Stage 2:** After receiving a prompt, route it to the best model within the selected pool.

Prompts are modeled as i.i.d. draws from an unknown deployment distribution $\mathcal{D}$. We approximate the expectation via Sample Average Approximation (SAA) over the 240 observed prompts.

Research Questions. My two research questions are listed below.

1. **Q1 (Pool Size Efficiency):** How large must the model pool K be to achieve near-optimal performance? Do we observe diminishing marginal returns as K increases?
2. **Q2 (Distribution Robustness):** Is the optimal pool composition robust to deployment distribution shifts across benchmark domains?

2 Optimization Model

2.1 Notation

Sets and indices:

- $\mathcal{P} = \{p\}_{p=1}^{240}$: set of 240 sampled prompts (60 per benchmark).

- $\mathcal{M} = \{m\}_{m=1}^{33}$: set of 33 candidate LLM models.
- $\mathcal{A} \subseteq \mathcal{P} \times \mathcal{M}$: valid (prompt, model) pairs with observed data ($|\mathcal{A}| = 7,860$).
- $\mathcal{D} = \{\text{AIME, GPQA, LCB, MMLU-Pro}\}$: benchmark datasets; $d(p) \in \mathcal{D}$ is the benchmark of prompt p .
- $\mathcal{M}^0 \subseteq \mathcal{M}$: zero-cost open-source models; $|\mathcal{M}^0| = 9$.

Parameters:

- $w_p \in [0, 1]$, $\sum_{p \in \mathcal{P}} w_p = 1$: empirical prompt weight; $w_p = 1/240$ under uniform SAA.
- $C(p, m) \geq 0$: API cost to route prompt p to model m ; $C(p, m) = 0$ for all $m \in \mathcal{M}^0$.
- $Q(p, m) \in \{0, 1\}$: binary correctness of model m on prompt p (1 = correct).
- $\alpha > 0$: trade-off weight balancing quality gain against cost.
- $K \in \mathbb{Z}_+$: maximum pool size.
- $B \geq 0$: budget cap on the weighted average API cost per prompt.
- $q_{\min} \in [0, 1]$: minimum required weighted average correctness score (quality floor).

Decision variables:

- $y_m \in \{0, 1\}$, $m \in \mathcal{M}$: Stage 1 binary indicator, equals 1 if model m is selected into the pool, decided before any prompt is seen.
- $x_{pm} \in \{0, 1\}$, $(p, m) \in \mathcal{A}$: Stage 2 binary indicator, equals 1 if prompt p is routed to model m , decided after observing the prompt.

2.2 Population Form

Let $\xi \sim \mathcal{D}$ be a random prompt. Stage 1 minimizes the expected Stage-2 objective over all realizations

$$\min_{y \in \{0,1\}^{|\mathcal{M}|}} \mathbb{E}_\xi [Q_2(y, \xi)], \quad (1)$$

where the Stage-2 value function solves, for each realization ξ :

$$\begin{aligned} Q_2(y, \xi) = & \min_{x(\xi) \in \{0,1\}^{|\mathcal{M}|}} \sum_{m \in \mathcal{M}} C(\xi, m) x_m(\xi) - \alpha \sum_{m \in \mathcal{M}} Q(\xi, m) x_m(\xi) \\ \text{s.t. } & \sum_{m \in \mathcal{M}} x_m(\xi) = 1, \quad x_m(\xi) \leq y_m \quad \forall m \in \mathcal{M}. \end{aligned} \quad (2)$$

The first Stage-2 constraint ensures every prompt is routed to exactly one model. The linking constraint (C2) restricts routing to models already selected in Stage 1.

2.3 SAA Approximation and Solvable Mixed Integer LP

Since $\mathcal{D}$ is unknown, we replace $\mathbb{E}_\xi[\cdot]$ with the empirical mean. This gives:

$$\min_{y, x} \sum_{(p, m) \in \mathcal{A}} w_p C(p, m) x_{pm} - \alpha \sum_{(p, m) \in \mathcal{A}} w_p Q(p, m) x_{pm} \quad (3)$$

subject to the six constraints in Table 1.

2.4 Linearization

Formulation (3) is already fully linear in the decision variables. Because $C(p, m)$ and $Q(p, m)$ are fixed data parameters, the objective is linear in x_{pm} . Therefore, the model is a pure integer linear program requiring no auxiliary variables.

Table 1: MILP constraints with scope and business motivation.

Label	Constraint	Scope	Description
(C1)	$\sum_{m: (p,m) \in \mathcal{A}} x_{pm} = 1$	$\forall p \in \mathcal{P}$	Each prompt routed to exactly one model
(C2)	$x_{pm} \leq y_m$	$\forall (p,m) \in \mathcal{A}$	Route only to pool models, linking stage 1 and 2
(C3)	$\sum_{m \in \mathcal{M}} y_m \leq K$	—	Limiting pool size
(C4)	$\sum_{(p,m) \in \mathcal{A}} w_p C(p,m) x_{pm} \leq B$	—	Budget cap on average cost per prompt
(C5)	$\sum_{m \in \mathcal{M}^0} y_m \geq 1$	—	At least one free fallback model in pool
(C6)	$\sum_{(p,m) \in \mathcal{A}} w_p Q(p,m) x_{pm} \geq q_{\min}$	—	Minimum quality floor
$y_m \in \{0,1\}, x_{pm} \in \{0,1\}$			Binary integrality

2.5 Reweighted SAA for Distribution (RQ2)

To model domain-specific prompt distributions, we replace the original uniform weights with

$$w_p^{(\text{boost})} = \frac{\tilde{w}_p}{\sum_{p' \in \mathcal{P}} \tilde{w}_{p'}}, \quad \tilde{w}_p = \begin{cases} \beta & \text{if } d(p) \in \mathcal{F}, \\ 1 & \text{otherwise,} \end{cases} \quad (4)$$

with boost factor $\beta = 3$ and focus set $\mathcal{F} \subseteq \mathcal{D}$. We are looking at four domain categories: Uniform ($\mathcal{F} = \emptyset$), Coding-Heavy ($\mathcal{F} = \{\text{LCB}\}$), Math-Heavy ($\mathcal{F} = \{\text{AIME}\}$), Knowledge-Heavy ($\mathcal{F} = \{\text{GPQA}, \text{MMLU-Pro}\}$).

3 Pyomo Implementation

The model is implemented in `RouteMind.ipynb` as `solve_two_stage_router` function. The code is available in the GitHub repository: https://github.com/ichen22/RouteMind_AI.

We got the optimal solution at $\alpha = 1.0$, $K = 6$, with uniform weights, weighted average score = 0.983, weighted average cost = \$0.00741 per prompt.

4 Results and Visualization

All experiments use $\alpha = 1.0$ and uniform SAA weights unless noted. $\alpha = 1.0$ is the lowest value at which $K=6$ first achieves oracle score, making it the most informative for RQ1. It also places cost and score on comparable scales so neither dominates. Figures are saved to `outputs/`.

4.1 Data Exploration

Three findings motivated the two-stage approach: **(i)** models disagree per prompt with mean cross-model std ≈ 0.44 , so per prompt routing be better over any fixed single model; **(ii)** cost–score correlation is only 0.48, meaning that expensive models are not always better (Figure 1, right); **(iii)**

18 distinct models win on at least one prompt (Figure 2), so no single model dominates across the full distribution.

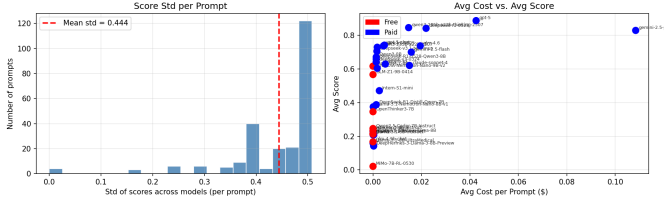


Figure 1: Left: score std per prompt (mean ≈ 0.44). Right: avg cost vs. avg score per model. Red = free. Cost–score correlation = 0.48.

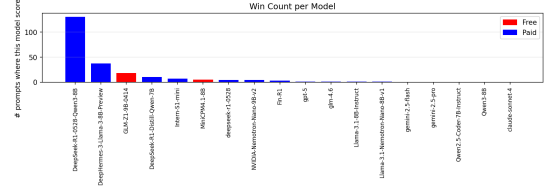


Figure 2: Win count per model. 18 models each win on at least one prompt.

4.2 RQ1: Pool Size Efficiency

Figure 3 shows K from 1 to 12 at $\alpha = 1.0$. The score curve is concave with diminishing returns. The model first achieves oracle score at $K = 6$ (score = 0.983, cost = \$0.00741). Beyond $K = 6$, larger K does not improve score but reduces cost. Thus $K = 6$ is recommended when operational simplicity (fewer model) is prioritized over cost.

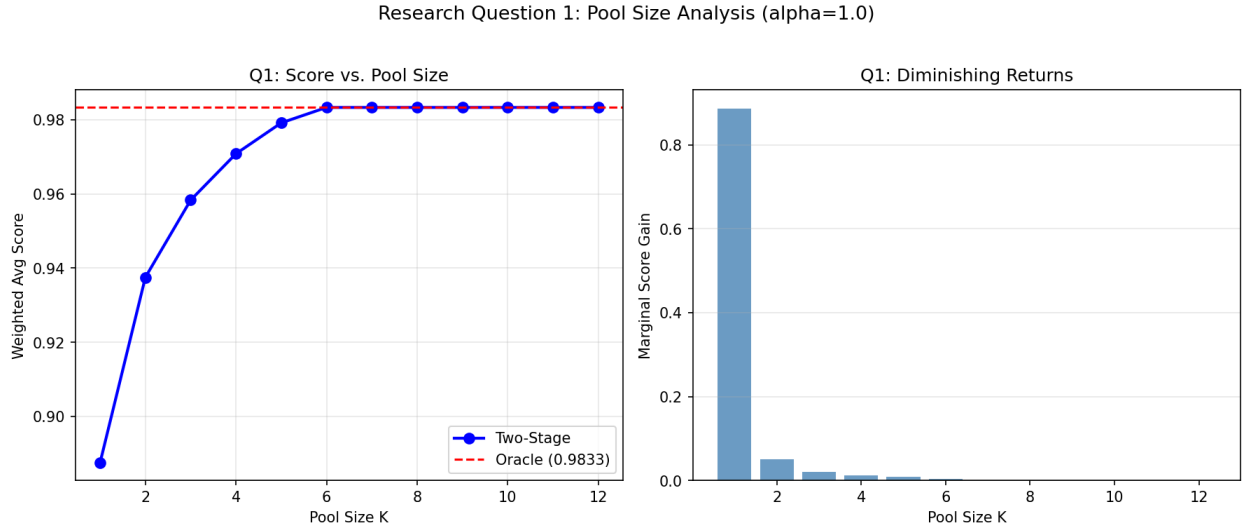


Figure 3: Q1: score vs. pool size K (left) and marginal gain per additional model (right). Oracle score is first reached at $K = 6$; beyond that, additional models yield no score improvement.

4.3 RQ2: Distribution Robustness

Figure 4 shows cost–performance frontiers across the four weight schemes at $K = 6$.

- **Pool composition is stable.** All four reweighted distributions selected the same six models at $K = 6$. (GLM-Z1-9B-0414, Llama-3.1-8B-UltraMedical, gemini-2.5-flash, glm-4.6, gpt-5, kimi-k2-0905)
- **Frontiers diverge by domain.** Even with the same pool, the achievable score varies. This reflects AIME’s inherent difficulty and the fact that mathematical reasoning benchmark is hardest to cover even with the best available models.

Since the same pool is optimal for every distribution, the uniform-weight policy is sufficient for pool selection. The frontier divergence is inherent to benchmark difficulty, not a limitation of the policy.

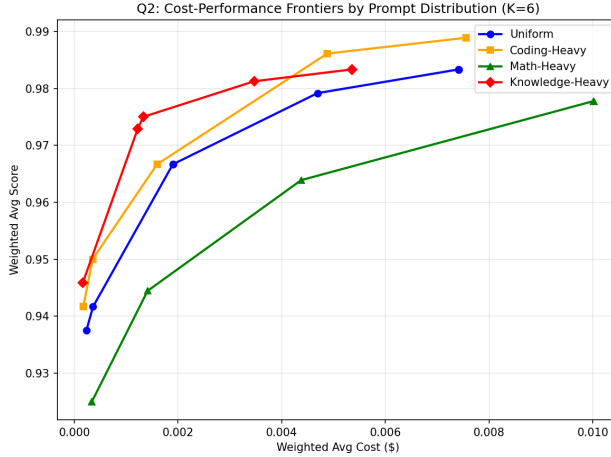


Figure 4: Q2: cost-performance frontiers by weight scheme at $K = 6$.

4.4 Constraint Ablation and Parameter Sweeps

Ablation (Figure 5). Starting from the unconstrained baseline ($K = 33$, score = 0.983, cost = \$0.00168), we first add C3 (pool size $K = 6$) and then add each remaining constraint on top of C3:

Configuration	Score	Cost (\$/prompt)	Note
No extra constraints ($K = 33$)	0.983	0.00168	Baseline
+C3: Pool size $K = 6$	0.983	0.00741	Oracle score maintained; $+4.4\times$ cost
+C4: Budget $B = \$0.005$	0.979	0.00469	-0.004 score, -37% cost vs. C3
+C5: Min 1 free model	0.983	0.00741	Non-binding (free model already in pool)
+C6: Quality floor $q_{\min} = 0.68$	0.983	0.00741	Non-binding ($0.983 \gg 0.68$)
Full model (C3+C4+C5)	0.979	0.00469	Practical recommended config

C4 delivers the most cost reduction (-37%) at a modest -0.4% score penalty. C5 and C6 are non-binding at the chosen parameters, confirming the $K=6$ pool is naturally robust.

Budget Sensitivity. At $K = 6$, the budget constraint becomes binding below $B = \$0.00741$. Analysis of B reveals that budgeting cost to $B = \$0.005$ saves 37% of cost at only a -0.4% score loss, while tightening it further to $B = \$0.002$ saves an additional 60% of remaining cost but at a -1.3% additional score loss, which is a much worse tradeoff. Thus $B = \$0.005$ is a good budget when cost control is needed.

Quality Threshold. At $K = 6$ and $\alpha = 1.0$, all tested $q_{\min} \in [0.50, 0.74]$ are non-binding: the two-stage $K=6$ solution inherently achieves score = 0.983, so quality floors up to 0.74 come at zero cost penalty.

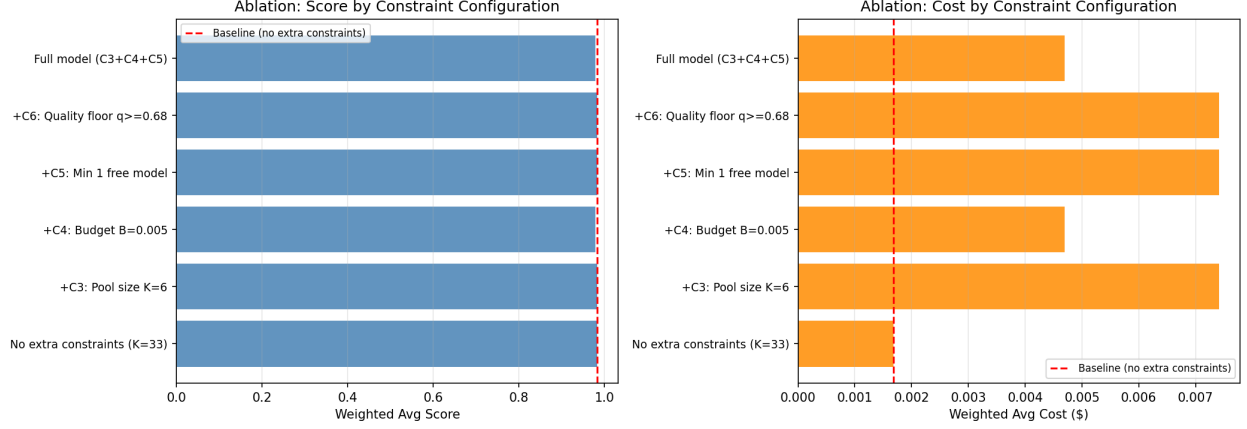


Figure 5: Constraint ablation: score (left) and cost (right). C5 and C6 are non-binding at these parameters; C4 achieves the sharpest cost reduction.

4.5 Full Cost–Performance Frontier

Figure 6 places all six policies on the cost–performance plane across the full α grid. Two-Stage $K = 6$ and $K = 10$ cluster tightly in the upper-left corner, confirming that a small optimized pool matches oracle routing. $K = 3$ peaks at 0.958 and never reaches oracle score regardless of α , showing that below the threshold pool size, quality is fundamentally limited.

Table 2 summarizes performance at $\alpha = 1.0$.

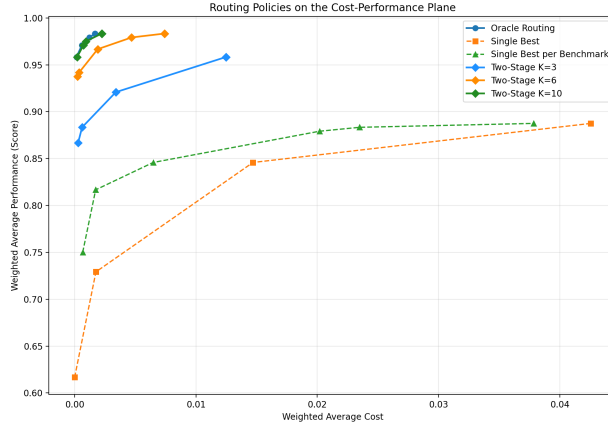


Figure 6: Two-Stage $K=10$ overlaps the oracle; $K=6$ is the first pool size to reach oracle score.

5 Discussion and Recommendations

Q1. How does your model answer the research questions? The pool-size shows strong and early diminishing marginal returns: the marginal score gain drops from 0.050 at $K = 1 \rightarrow 2$ to 0.004 at $K = 5 \rightarrow 6$, and falls to zero for $K \geq 6$. Beyond $K = 6$, score cannot improve further, but cost continues to fall ($K = 10$ reaches \$0.00224, a 70% saving over $K = 6$).

Rewighted SAA shows that pool composition is robust but frontier performance is not. All four domain distributions select the same six-model pool at $K = 6$, confirming the uniform-weight policy

Table 2: Policy summary at $\alpha = 1.0$, uniform weights.

Policy	Avg Score	Avg Cost (\$/prompt)	% Oracle Score
Oracle Routing ($K = 33$)	0.983	0.00168	100.0%
Two-Stage $K = 10$	0.983	0.00224	100.0%
Two-Stage $K = 6$	0.983	0.00741	100.0%
Two-Stage $K = 3$	0.958	0.01248	97.5%
Single Best (gpt-5)	0.888	0.04255	90.3%
Single Best per Benchmark	0.883	0.02350	89.8%

is universally optimal for pool selection. However, the achievable score varies across distributions: Coding-Heavy reaches 0.976 while Math-Heavy reaches only 0.961, reflecting AIME’s inherent difficulty.

Q2. What business recommendations do you want to convey to your manager?

1. **Choose $K = 6$ for operational simplicity.** It is the minimum pool size that achieves oracle score (0.983), with only 6 model integrations to manage.
2. **Choose $K = 10$ for API cost efficiency.** It achieves the same score (0.983) at \$0.00224/prompt (a 70% cost reduction over $K = 6$) with 4 additional model integration.
3. **For $K = 6$, apply a budget cap $B \approx \$0.005$** when cost control is required. This reduces cost by 37% (to \$0.00469) at a score penalty of only -0.4% (from 0.983 to 0.979).

Q3. What further improvements or extensions would you wish to attempt if you have more time? One improvement would be to incorporate real-time latency data with cost and correctness, so the router can trade off speed against quality rather than cost alone. This would make the model more realistic for production settings where response time directly affects user experience.

One extension I wish to attempt is to include a semantic analysis of the prompts if I had access to the actual prompt text. Rather than just relying on benchmark domain labels, we could embed each prompt using a text embedding model and compute pairwise cosine similarities. Prompts that share high cosine similarity in embedding space likely require the same type of reasoning, so they could be routed to the same model.

6 Personal Takeaway

As someone who is interested in having a full-time AI/ML role in the future, this project was very interesting. Since this project is a real-world problem with actual benchmarks I’ve used for evaluating LLMs in my past internships, it helped me understand more about how these automatic LLM routing programs within AI products might work. Thank you for a great course this semester, Professor and course staff!